A Phase-Aware Adaptive Bank Mapping Approach for Reducing GPU Shared Memory Bank Conflicts

Devaananth P
Department of Computer Science and Engineering
SRM Institute of Science and Technology
Chennai, India
devaananthmsd31@gmail.com

Abstract— Shared-memory bank conflict is a major problem in GPUs and occurs when threads in a warp access different addresses mapped to the same memory bank. In this work, we study an adaptive mapping method for workloads where the access patterns change during runtime. The controller compares Normal, XOR-A, and XOR-B mappings and changes the active mapping when another mapping gives a lower conflict cost. To preserve data correctness, switching is allowed only at a safe data-lifetime boundary. The switches that are deferred are checked again before they are applied.

The controller was developed and tested through 35 experiments with different access patterns and conditions. During these experiments, several versions of the controller were tested before reaching the final lifetime-safe design. In the Phase-Mixed workload, the best static mapping gave a conflict cost of 3296, while the lifetime-safe adaptive controller reduced it to 1559. This gives a reduction of 52.70%, with no unsafe remapping during the test. We also tested the effect of bank conflicts on real GPUs using an RTX 5070 and Tesla T4. In the matrix-transpose test, shared-memory padding reduced the runtime by 26.38% on the RTX 5070 and 52.78% on the Tesla T4, while both versions produced the same output. The adaptive controller was tested using simulation, while the GPU experiments were used to check how reducing bank conflicts affects the actual execution time.

Keywords— GPU shared memory, bank conflicts, adaptive bank mapping, phase-aware optimization, runtime remapping, CUDA

I. Introduction

GPUs execute a large number of threads in parallel, which makes them suitable for workloads that require high computational performance. Along with global memory, GPUs provide shared memory that can be accessed by threads within the same block. Shared memory is faster than global memory, but its performance also depends on how the threads access it. The memory is divided into multiple banks so that several accesses can be handled at the same time. A problem occurs when multiple threads in a warp access different addresses that are mapped to the same bank. These accesses cannot be completed together and have to be handled in multiple steps, which creates a bank conflict. The number of conflicts depends on the memory access pattern of the workload. Some access patterns can work well with a particular bank mapping, while the same mapping can give more conflicts when the pattern changes. Because of this, using a single static mapping for the complete execution may not always give the lowest conflict cost. This became the main reason for studying an adaptive mapping method that can change the active mapping based on the current access pattern. The main contribution of this work is a phase-aware adaptive bank-mapping controller that can change the selected mapping when the access pattern changes during execution. The controller also allows mapping changes only at safe data-lifetime boundaries to protect previously stored shared-memory data. If a mapping change is deferred, the decision is checked again at the safe boundary before it is applied.

II. Related Work

Some techniques aim to hide the delay that bank conflicts create while others work on altering how addresses are mapped to memory banks. Gou and Gaydadjiev [1] came up with an elastic pipeline method that reduces the impact of bank conflicts while a program runs. Their study mainly tackles the delay from bank conflicts, not the mapping itself. Another key direction is XOR-based bank mapping. Van den Braak et al [2] examined XOR hash functions for GPU scratchpad memories. The goal is to spread accesses across banks by changing which address bits pick a bank. Their experiments showed that this mapping can cut bank conflicts a lot when the access patterns fit the design. The research similar to this paper comes from Han et al [3] who suggested an access-pattern-based adaptive mapping for GPGPU scratchpad memory. Their method gathers access data while the first thread block runs, then **selects** a mapping for later execution**.** This shows that adaptive bank mapping can help**.** The chosen mapping relies mainly on past behaviour. What sets this work apart is its focus. Rather than assuming a single mapping stays good after the first decision the new method accepts that access patterns may shift during execution. It also introduces remapping points to keep the mapping steady while old shared-memory data stays active. Deferred decisions are re-examined before use, helping to avoid a mapping that no longer fits the access pattern.

III. PROPOSED METHODOLOGY

The proposed method uses an adaptive controller to change the bank mapping when the memory access pattern changes during execution. Instead of using one mapping for the complete workload, the controller has three possible mappings which are Normal, XOR-A and XOR-B. The

reason for using multiple mappings is that the same address pattern can be distributed between the memory banks differently depending on the mapping. A mapping which gives fewer conflicts for one access pattern may give more conflicts after the pattern changes. The controller checks the memory accesses during execution and compares the three mappings before deciding whether the current mapping should be changed. This makes the mapping decision depend on the current access behaviour instead of selecting one mapping at the beginning and keeping it for the complete workload. The controller also controls when the mapping can be changed because frequent switching is not always useful and changing the mapping while old shared-memory data is still active can create a correctness problem.

The three mappings use different address bits for selecting one of the 32 memory banks. In the Normal mapping, the bank is selected directly from the lower part of the address. In the simulator this is calculated as

$$\text{Normal} = \text{address} \bmod 32$$

For the XOR-A mapping, two groups of five address bits are used. The lower five bits and the next five bits are calculated as

$$\text{low5} = \text{address} \,\&\, 31$$
$$\text{high5} = (\text{address} \gg 5) \,\&\, 31$$

and the bank is selected using

$$\text{XOR-A} = \text{low5} \oplus \text{high5}$$

XOR-B uses the same lower five bits but changes the second group of bits:

$$\text{mid5} = (\text{address} \gg 3) \,\&\, 31$$
$$\text{XOR-B} = \text{low5} \oplus \text{mid5}$$

The purpose of XOR-A and XOR-B is to distribute the same memory addresses between the banks in a different way from the Normal mapping. Because of this, an access pattern that creates several conflicts with one mapping can give a lower conflict cost with another mapping. These are simplified mapping models used for testing the controller and they do not represent the exact internal bank mapping used by a real NVIDIA GPU. For every memory access pattern, the controller checks how the 32 threads are distributed between the 32 banks under each mapping. When different addresses from multiple threads are mapped to the same bank, they contribute to the conflict cost. A mapping that distributes the accesses across more banks will normally give a lower conflict cost than a mapping where many of the accesses fall into the same banks. The controller calculates this cost separately for Normal, XOR-A and XOR-B using the upcoming memory accesses. The three values are then compared to find which mapping gives the lowest expected conflict cost. The selected mapping is also compared with the mapping which is currently active. If the current mapping is already the best choice, no change is requested. If another mapping gives a lower conflict cost, the controller can request a mapping switch. The conflict cost used here is an experimental metric for comparing the mappings in the simulator. It is not treated as GPU execution time, clock cycles or energy consumption. This is important because the simulator is used to study the mapping behaviour, while the actual execution-time effect of bank conflicts is tested separately using the real-GPU experiments.

Selecting the mapping with the lowest conflict cost is not enough because the controller can start changing the mapping too often. During the experiments we found that a small change in the upcoming accesses can make another mapping look better for a short time. If the controller switches immediately, the access pattern may change again after a few instructions and another switch may be requested. To reduce these unnecessary switches, a minimum hold time and a switching threshold are used. The minimum hold time keeps the selected mapping active for a short period before another normal switch is allowed. The threshold is used to make sure that another mapping gives enough improvement before the controller changes from the current mapping. This prevents the controller from switching only because of a small difference in conflict cost. Harder access patterns also showed cases where the controller could remain on a poor mapping after the workload moved into another phase. A causal watchdog was added to check this behaviour using the accesses that have already occurred. An emergency response is also used when the controller needs to recover from a poor mapping instead of waiting for the normal switching condition. These parts help the controller react to phase changes while still controlling the number of mapping switches.

Another problem appears when the controller changes the mapping while shared-memory data is still active. The mapping decides which bank is used for an address. If some data is written using one mapping and the controller changes to another mapping before that data is no longer required, a later read can use a different bank mapping from the earlier write. This can result in incorrect data being accessed. Because of this, the final controller does not apply a requested mapping switch immediately. The switch is allowed only when the execution reaches a safe data-lifetime boundary where the old shared-memory data is no longer required. If a safe boundary is available, the controller can apply the new mapping without affecting data stored under the previous mapping. If the controller requests a switch when there is no safe boundary, the switch is kept as a deferred decision. During the experiments we found that directly applying this decision at the next boundary can also create a problem. The access pattern may have changed while the controller was waiting, so the mapping selected earlier may no longer give the lowest conflict cost. The final controller therefore checks the deferred decision again when the safe boundary is reached. Normal, XOR-A and XOR-B are evaluated again using the current access behaviour. The switch is made only if the deferred mapping is still useful. Otherwise, the old

decision is not applied. This allows the controller to change its mapping during execution while preventing a mapping change from being made using an outdated decision. The complete controller therefore works in a sequence of steps. First, the upcoming memory accesses are checked with Normal, XOR-A and XOR-B and their conflict costs are compared. If the current mapping is still suitable, the controller continues without making a change. If another mapping gives a better result, the switching conditions such as the minimum hold time and switching threshold are checked. The watchdog and emergency response can also affect the decision when the current mapping continues to perform poorly. After a switch is requested, the controller checks whether a safe data-lifetime boundary is available. If the boundary is available, the new mapping can be applied. If it is not available, the decision is deferred. When the next safe boundary is reached, the mapping decision is calculated again instead of directly applying the old request. The controller then either changes to the new mapping or continues with the current mapping depending on the new conflict-cost comparison. This process is repeated as the workload continues. In this way, the controller can respond when the preferred mapping changes between different phases while also controlling unnecessary switches and preventing unsafe remapping of active shared-memory data.

IV. EXPERIMENTAL SETUP

The experiments were done in two parts. The first part used our simulator to develop and test the adaptive bank-mapping controller. The second part used CUDA kernels on real GPUs to check how bank conflicts affect the actual execution time. The real-GPU experiments were kept separate from the adaptive-controller simulation because CUDA does not allow us to replace the physical bank-mapping function of the GPU with Normal, XOR-A and XOR-B. For the simulation, 32 memory banks and 32 threads in a warp were used. Each memory access contains the addresses generated by the threads in the warp. The same addresses are tested using Normal, XOR-A and XOR-B so that their conflict costs can be compared. The workload is also executed separately using each mapping as a static mapping. This gives the best static result for the same workload and allows us to compare it with the adaptive controller. Different access patterns were used because the main purpose of the controller is to handle cases where the preferred mapping can change during execution. The tests included stable patterns, short phases, rapidly changing patterns, random and irregular accesses and workloads containing multiple access phases. We also used access patterns derived from matrix transpose, parallel reduction and prefix scan. A Phase-Mixed workload was used to specifically test a case where one mapping may work well during one part of the execution but another mapping can become better after the access pattern changes.

The adaptive controller was developed through 35 experiments. We did not use only one type of workload during this development because a controller that works well for a stable pattern can behave differently when the access pattern changes frequently. During these experiments we tested different lookahead behaviour, switching thresholds, minimum hold times, fallback methods, watchdog logic, emergency recovery and finally the safe-remapping mechanism. A larger stress test was also used during the development of the controller. It contained 200 workloads with 2000 memory-access instructions in each workload. The access patterns included Stable, Micro Phase, Adversarial, Random Stride, Irregular, Clustered and Chaotic Mixed cases. Different strides such as 1, 2, 3, 4, 5, 7, 8, 12, 16, 24, 32, 48 and 64 were used to create different ways of distributing the accesses between the banks. These workloads helped us find cases where the current mapping could become poor after the access pattern changed. Lookahead was also tested under different availability conditions during controller development. In one of the hard synthetic datasets, the average available lookahead was 65.28%. This was used to check how the controller behaves when all the upcoming accesses are not available. For the final deterministic validation, full lookahead was used so that the mapping methods and the lifetime-safe controller could be compared under the same condition. For the simulation experiments, we recorded the conflict cost, number of mapping switches and unsafe remapping attempts. Correctness was also checked during the development of the lifetime-safe controller. These values were used to compare the different controller versions and to identify cases where reducing the conflict cost could introduce too many switches or an unsafe mapping change.

For the hardware experiments, CUDA kernels were executed on an NVIDIA GeForce RTX 5070 and a Tesla T4. CUDA events were used to measure the kernel execution time. Each experiment was first executed for 10 warm-up runs so that the initial execution did not affect the measured result. After the warm-up, 50 runs were measured and these values were used for the hardware results. The first hardware experiment was a phase-changing shared-memory bank-conflict stress test. It used 256 blocks with 32 threads in each block. The kernel used 100 phase repetitions with 100 iterations in each phase. The shared-memory access stride was changed using the sequence 1, 2, 4, 8, 16, 32, 16, 8, 4, 2 and 1. This creates different levels of bank conflict during the same kernel execution instead of using only one fixed access pattern. A conflict-free reference kernel was also executed for comparison. In the reference case, shared memory was accessed using the thread ID. In the phase-changing version, the shared-memory index was generated using the thread ID and the current stride. Both versions were tested using the same timing procedure on the RTX 5070 and Tesla T4. The mean execution time was then used to calculate the slowdown caused by the phase-changing conflict pattern. The second hardware experiment used matrix transpose to test a common method for reducing shared-memory bank conflicts without changing the actual computation. A 4096 × 4096 matrix was used for this experiment. The baseline kernel used a 32 × 32 shared-memory tile, which was declared as tile[32][32]. The padded version used tile[32][33]. Adding one extra column changes how the

elements are placed between the shared-memory banks and can reduce the conflicts that occur during the transpose operation. Both kernels performed the same matrix-transpose operation. Only the shared-memory layout was changed so that the performance difference could be compared without changing the required output. The baseline and padded kernels were executed in alternating order to reduce the effect of one version always being executed first. As with the previous hardware experiment, 10 warm-up runs were followed by 50 measured runs on both the RTX 5070 and Tesla T4. The output produced by the baseline and padded kernels was also compared after execution. The number of output mismatches was recorded to make sure that changing the shared-memory layout did not change the result of the transpose operation. The execution times from the two versions were then used to calculate the runtime reduction and speedup obtained from the padded layout. The real-GPU experiments are used as a separate hardware validation of the bank-conflict problem. The Normal, XOR-A and XOR-B adaptive mappings are not directly executed on the RTX 5070 or Tesla T4 because CUDA does not provide control over the physical bank-mapping function of the GPU. Because of this, the runtime improvements from the hardware experiments should not be treated as the hardware performance of the proposed adaptive controller. Instead, the hardware tests are used to check whether changing the shared-memory access behaviour and reducing bank conflicts can produce a measurable difference in actual execution time. The simulator is used to evaluate the proposed adaptive mapping method, while the CUDA experiments are used to study the performance effect of bank conflicts on real GPU hardware.

The source code and validation scripts used in this work are available at: https://github.com/Devaananth357/phase-aware-gpu-bank-mapping.

V. RESULTS AND DISCUSSION

The final controller was tested with matrix transpose, parallel reduction, prefix scan and the Phase-Mixed workload. Table I compares the best static mapping with the unrestricted adaptive controller and the final lifetime-safe controller. For matrix transpose, all three methods gave the same conflict cost of 128. The same behaviour can be seen in parallel reduction and prefix scan, where the conflict costs remained at 768 and 1536. In these workloads, the access pattern does not require frequent changes between the mappings, so the adaptive controller does not give an additional reduction compared with the best static mapping. The main difference can be seen in the Phase-Mixed workload. The best static mapping produced a conflict cost of 3296, while the unrestricted adaptive controller reduced it to 2353. The lifetime-safe controller gave the lowest conflict cost of 1559. This is a 52.70% reduction compared with the best static mapping.

Benchmark	Best Static	Unrestricted Adaptive	Lifetime-safe Adaptive
Matrix Transpose	128	128	128
Parallel Reduction	768	768	768
Prefix Scan	1536	1536	1536
Phase-Mixed	3296	2353	1559

Table I. Conflict cost comparison between static and adaptive mapping methods.

Figure 1 shows the same results as a graph, which makes the difference in the Phase-Mixed workload easier to see. For the first three workloads, the bars remain at the same level because there is no improvement from changing the mapping. The difference becomes much larger for Phase-Mixed. This happens because Phase-Mixed contains different access phases and the mapping which works well for one phase may not remain suitable for the next phase.

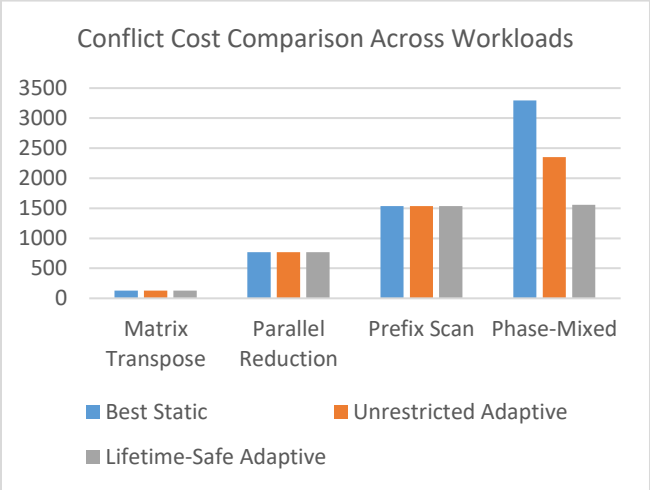


Fig. 1. Conflict cost comparison between static and adaptive bank mapping methods.

The result from Phase-Mixed is important for understanding where the proposed method is useful. The adaptive controller is not expected to reduce conflicts when the same mapping already works well for the complete workload. Its advantage comes when the access behaviour changes during execution. In this case, the controller can move to another mapping instead of continuing with a static choice. The lifetime-safe version also completed the final test with zero unsafe remapping attempts, so the reduction in conflict cost was achieved without allowing a mapping change while old shared-memory data was still active. We tested different versions of the controller before selecting the final method. In the starting experiments, previous memory accesses were used for selecting the mapping. This worked for some patterns, but when the access pattern changed the controller was not able to change the mapping at the correct time. So we tested lookahead where the upcoming accesses are checked before selecting between Normal, XOR-A and XOR-B. This gave better results, but another problem came where the controller started switching between the mappings too many times. For reducing these switches, we added a minimum hold time

and a threshold before allowing the mapping to change. After this we tested the controller with harder workloads including short phases, irregular patterns and frequently changing accesses. In some cases the controller was staying in a bad mapping even after the access pattern had changed. We added a watchdog to find these cases and later an emergency response was used when the controller needed to move out of a bad mapping quickly. The next problem we found was related to correctness. We cannot simply change the mapping when shared-memory data from the previous mapping is still active. The read and write can then use different mappings. So we allowed the switches only at safe data-lifetime boundaries. If there is no safe boundary when the controller requests a switch, that decision is kept for later. We also found that blindly applying this old decision was not correct because the access pattern could already be different by the time the safe boundary comes. In the final controller, the deferred mapping is checked again at the boundary and the switch is made only if that mapping is still useful.

Controller Stage	Change Made	What We Found
Initial Adaptive	Used previous access history to select the mapping	Worked for stable patterns but had problems when the pattern changed
Lookahead	Checked upcoming memory accesses before selecting the mapping	Gave better decisions but caused too many mapping switches
Switch Control	Added minimum hold time and switching threshold	Reduced unnecessary switches between mappings
Hybrid Controller	Combined lookahead with fallback behaviour	Improved the controller on harder and changing workloads
Causal Watchdog	Checked whether the current mapping was performing poorly	Helped identify a bad mapping during execution
Emergency Response	Allowed faster recovery from a bad mapping	Reduced the time spent using a poor mapping
Lifetime-Safe Switching	Allowed switches only at safe data-lifetime boundaries	Prevented mapping changes while old shared-memory data was active
Deferred Switching	Stored a switch when a safe boundary was not available	Maintained correctness, but the decision could become outdated
Final Controller	Rechecked the deferred decision at the next safe boundary	Avoided using an old mapping decision after the access pattern changed

Table II. Main stages in the development of the adaptive controller

Table II shows the main changes we made while developing the controller. The first versions were mainly focused on finding which mapping gives the lower conflict cost. During the experiments we found that selecting the mapping alone was not enough. The controller also had to control how often it switches and how it reacts when the access pattern changes suddenly.

The bigger problem came when we started considering the data stored in shared memory. Changing the mapping while this data is still active can affect the later memory accesses. So we restricted the mapping changes to safe data-lifetime boundaries. When a switch cannot be made immediately, it is kept as a deferred decision. We later found that this decision can become old if the access pattern changes before reaching the boundary. Because of this, the final controller checks the mapping again at the safe boundary before making the switch.

After testing the controller in simulation, we also tested the effect of bank conflicts on real GPU hardware. For this test, the access stride was changed during execution to create different levels of bank conflicts. A conflict-free version was also executed as the reference. The same experiment was tested on the RTX 5070 and Tesla T4. Table III shows the mean execution time and slowdown measured on both GPUs.

GPU	Conflict-Free Mean (ms)	Phase-Mixed Mean (ms)	Slowdown
RTX 5070	1.8169	4.4465	2.4473×
Tesla T4	3.0667	15.5467	5.0695×

Table III. Execution time of conflict-free and Phase-Mixed access patterns on real GPUs

The results show a clear increase in execution time when bank conflicts are introduced. On the RTX 5070, the mean execution time increased from 1.8169 ms to 4.4465 ms, giving a slowdown of 2.4473×. On the Tesla T4, it increased from 3.0667 ms to 15.5467 ms, which gives a slowdown of 5.0695×. These results show that bank conflicts can produce a large runtime difference on actual GPU hardware and the effect can also be different between GPUs. The raw execution times between the RTX 5070 and Tesla T4 are not used to compare which GPU handles bank conflicts better because they are different GPU architectures. The important value here is the change between the conflict-free and Phase-Mixed versions on the same GPU.

We also tested a matrix-transpose kernel to check the effect of reducing bank conflicts without changing the output of the operation. Two versions of the kernel were used. The baseline version used a 32 × 32 shared-memory tile, while the second version used a 32 × 33 padded tile. The padding changes how the memory accesses are distributed between the banks. Both versions performed the same transpose operation and their outputs were compared after execution.

GPU	Baseline (ms)	Padded (ms)	Reduction
RTX 5070	0.4481	0.3299	26.38%
Tesla T4	1.4176	0.6695	52.78%

Table IV. Matrix-transpose performance with baseline and padded shared-memory layouts

Table IV shows that the padded version reduced the execution time on both GPUs. On the RTX 5070, the mean execution time decreased from 0.4481 ms to 0.3299 ms, giving a runtime reduction of 26.38% and a speedup of 1.3584×. On the Tesla T4, the execution time decreased from 1.4176 ms to 0.6695 ms, giving a reduction of 52.78% and a speedup of 2.1175×. Both tests produced zero output mismatches, showing that the padding did not change the result of the matrix transpose.

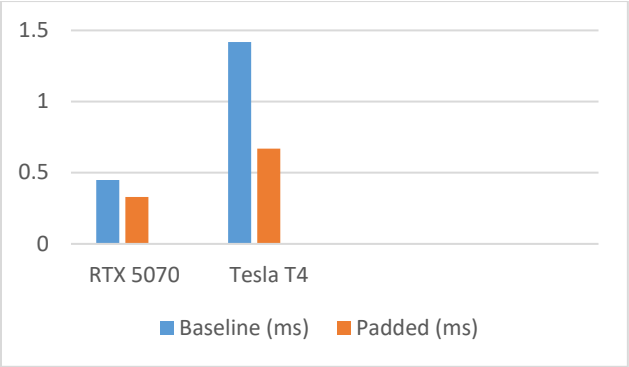


Fig. 2. Mean execution time of baseline and padded matrix-transpose kernels on real GPUs

Figure 2 shows the execution-time difference between the two shared-memory layouts. The padded version gives a lower execution time on both GPUs, with the larger reduction appearing on the Tesla T4.

VI. Limitations

The adaptive controller in this work was tested using simulation and was not implemented directly inside the bank-mapping hardware of a GPU. The Normal, XOR-A and XOR-B mappings used by the simulator are simplified mapping models and they do not represent the exact internal bank-mapping method used by current NVIDIA GPUs. Because of this, the conflict-cost reduction from the simulator should not be considered as the same as a hardware speedup. The real-GPU experiments were used separately to check the effect of bank conflicts on execution time. Shared-memory padding was used in the matrix-transpose experiment because CUDA does not allow us to directly replace the physical bank-mapping function with the proposed adaptive controller. The hardware results therefore show the performance effect of reducing bank conflicts, but they do not represent a hardware implementation of the proposed controller. Another limitation comes from safe remapping. The controller needs a safe data-lifetime boundary before changing the mapping because previously stored shared-memory data may still be required. When these safe boundaries are far apart, the controller has fewer chances to change the mapping even if another mapping gives a lower conflict cost. In this work the safe boundaries are provided by the experimental model. Detecting these boundaries automatically would be required for an actual hardware implementation. The Phase-Mixed workload also gives the largest improvement in the simulation, but it is a synthetic workload created to contain changing access phases. The 52.70% reduction in conflict cost should therefore be considered as a result from this experimental workload and not as a guaranteed improvement for every GPU workload.

VII. CONCLUSION

This work studied a phase-aware adaptive bank mapping method for GPU shared memory where the access pattern can change during execution. Instead of using one mapping for the complete workload, the controller selects between

Normal, XOR-A and XOR-B based on the current access behaviour. During the development of the controller, we also found that changing the mapping is not only a performance problem but also a data correctness problem. Because of this, the final method allows switching only at safe data-lifetime boundaries and checks the deferred mapping decision again before applying it. The controller was developed through 35 experiments with different access patterns and conditions. In the Phase-Mixed workload, the lifetime-safe controller reduced the conflict cost from 3296 to 1559 compared with the best static mapping, giving a reduction of 52.70%. The final test also completed without unsafe remapping attempts. The real-GPU experiments showed that bank conflicts can have a clear effect on execution time. In the matrix-transpose test, shared-memory padding reduced the mean runtime by 26.38% on the RTX 5070 and 52.78% on the Tesla T4, with zero output mismatches. These hardware results are separate from the adaptive-controller simulation, but they show that reducing shared-memory bank conflicts can give a measurable performance improvement. The next step for this work is to study how the controller can be implemented closer to actual GPU hardware and how safe data-lifetime boundaries can be detected automatically during execution.

VIII. References

[1] C. Gou and G. N. Gaydadjiev, “Addressing GPU on-chip shared memory bank conflicts using elastic pipeline,” International Journal of Parallel Programming, vol. 41, no. 3, pp. 400–429, 2013, doi: 10.1007/s10766-012-0201-1.

[2] G. J. van den Braak, J. Gómez-Luna, J. M. González-Linares, H. Corporaal, and N. Guil, “Configurable XOR hash functions for banked scratchpad memories in GPUs,” IEEE Transactions on Computers, vol. 65, no. 7, pp. 2045–2058, 2016, doi: 10.1109/TC.2015.2479595.

[3] F. Han, L. Li, K. Wang, F. Feng, H. Pan, J. Sha, and J. Lin, “An access pattern based adaptive mapping function for GPGPU scratchpad memory,” IEICE Electronics Express, vol. 14, no. 12, Art. no. 20170373, 2017, doi: 10.1587/elex.14.20170373.